

21 – Timing Programs, Counting Operations

Una introducción simplista a la complejidad algorítmica

Lo más importante que hay que considerar al diseñar e implementar un programa es que debe producir resultados en los que se pueda confiar. Queremos que los saldos de nuestros bancos se calculen correctamente. Queremos que los inyectores de combustible de nuestros automóviles inyecten cantidades apropiadas de combustible. Preferiríamos que ni los aviones ni los sistemas operativos se bloqueen.

A veces el rendimiento es un aspecto importante de la corrección. Esto es más obvio para los programas que necesitan ejecutarse en tiempo real. Un programa que advierte a los aviones sobre obstrucciones potenciales necesita emitir la advertencia antes de que se encuentren las obstrucciones. El rendimiento también puede afectar la utilidad de muchos programas que no son de tiempo real. El número de transacciones completadas por minuto es una métrica importante al evaluar la utilidad de los sistemas de bases de datos. Los usuarios se preocupan por el tiempo requerido para iniciar una aplicación en su teléfono. Los biólogos se preocupan por cuánto tiempo toman sus cálculos de inferencia filogenética.

Escribir programas eficientes no es fácil. La solución más directa a menudo no es la más eficiente. Los algoritmos computacionalmente eficientes a menudo emplean trucos sutiles que pueden hacerlos difíciles de entender. En consecuencia, los programadores a menudo aumentan la complejidad conceptual de un programa en un esfuerzo por reducir su complejidad computacional. Para hacer esto de manera sensata, necesitamos entender cómo estimar la complejidad computacional de un programa.

Pensando sobre la Complejidad Computacional

¿Cómo debería uno abordar la respuesta a la pregunta "¿Cuánto tiempo tomará ejecutarse la siguiente función?"

```
1 def f(i):
```

```
2     """Asume que i es un int e i >= 0"""
3     answer = 1
4     while i >= 1:
5         answer *= i
6         i -= 1
7     return answer
```

Podríamos ejecutar el programa con alguna entrada y cronometrarlo. Pero eso no sería particularmente informativo porque el resultado dependería de:

- La velocidad de la computadora en la que se ejecuta
- La eficiencia de la implementación de Python en esa máquina
- El valor de la entrada

Resolvemos los primeros dos problemas usando una medida más abstracta del tiempo. En lugar de medir el tiempo en microsegundos, medimos el tiempo en términos del número de pasos básicos ejecutados por el programa.

Para simplificar, usaremos una máquina de acceso aleatorio como nuestro modelo de computación. En una máquina de acceso aleatorio, los pasos se ejecutan secuencialmente, uno a la vez. Un paso es una operación que toma una cantidad fija de tiempo, como vincular una variable a un objeto, hacer una comparación, ejecutar una operación aritmética, o acceder a un objeto en memoria.

Ahora que tenemos una forma más abstracta de pensar sobre el significado del tiempo, pasamos a la cuestión de la dependencia del valor de la entrada. Tratamos con eso alejándonos de expresar la complejidad temporal como un número único y en su lugar relacionándola con los tamaños de las entradas. Esto nos permite comparar la eficiencia de dos algoritmos hablando sobre cómo el tiempo de ejecución de cada uno crece con respecto a los tamaños de las entradas.

Por supuesto, el tiempo de ejecución real de un algoritmo puede depender no solo de los tamaños de las entradas sino también de sus valores. Considera, por ejemplo, el algoritmo de búsqueda lineal implementado por

```
1     def linear_search(L, x):
2         for e in L:
3             if e == x:
4                 return True
5         return False
```

Supongamos que `L` es una lista que contiene un millón de elementos, y considera la llamada `linear_search(L, 3)`. Si el primer elemento en `L` es 3, `linear_search` retornará `True` casi inmediatamente. Por otro lado, si 3 no está en `L`, `linear_search` tendrá que examinar todo un millón de elementos antes de retornar `False`.

En general, hay tres casos amplios que considerar:

- El **tiempo de ejecución del mejor caso** es el tiempo de ejecución del algoritmo cuando las entradas son tan favorables como sea posible. Es decir, el tiempo de ejecución del mejor caso es el tiempo mínimo de ejecución sobre todas las posibles entradas de un tamaño dado. Para `linear_search`, el tiempo de ejecución del mejor caso es independiente del tamaño de `L`.
- De manera similar, el **tiempo de ejecución del peor caso** es el tiempo máximo de ejecución sobre todas las posibles entradas de un tamaño dado. Para `linear_search`, el tiempo de ejecución del peor caso es lineal en el tamaño de `L`.
- Por analogía con las definiciones del tiempo de ejecución del mejor caso y peor caso, el **tiempo de ejecución del caso promedio** (también llamado caso esperado) es el tiempo promedio de ejecución sobre todas las posibles entradas de un tamaño dado. Alternativamente, si uno tiene cierta información a priori sobre la distribución de los valores de entrada (por ejemplo, que 90% del tiempo `x` está en `L`), uno puede tomar eso en cuenta.

La gente usualmente se enfoca en el peor caso. Todos los ingenieros comparten un artículo común de fe, la Ley de Murphy: Si algo puede salir mal, saldrá mal. El peor caso proporciona una cota superior en el tiempo de ejecución. Esto es crítico cuando hay una restricción temporal sobre cuánto tiempo puede tomar un cálculo. No es suficientemente bueno saber que "la mayoría del tiempo" el sistema de control de tráfico aéreo advierte sobre colisiones inminentes antes de que ocurran.

Veamos el tiempo de ejecución del peor caso de una implementación iterativa de la función factorial:

```
1 def fact(n):
2     """Asume que n es un entero positivo
3     Retorna n!"""
4     answer = 1
5     while n > 1:
6         answer *= n
7         n -= 1
8     return answer
```

El número de pasos requeridos para ejecutar este programa es algo como 2 (1 para la declaración de asignación inicial y 1 para el return) + $5n$ (contando 1 paso para la prueba en el while, 2 pasos para la primera declaración de asignación en el bucle while, y 2 pasos para la segunda declaración de asignación en el bucle). Así, por ejemplo, si n es 1000, la función ejecutará aproximadamente 5002 pasos.

Debería ser inmediatamente obvio que mientras n se hace grande, preocuparse por la diferencia entre $5n$ y $5n+2$ es un poco tonto. Por esta razón, típicamente ignoramos las constantes aditivas al razonar sobre el tiempo de ejecución. Las constantes multiplicativas son más problemáticas. ¿Deberíamos preocuparnos por si el cálculo toma 1000 pasos o 5000 pasos? Los factores multiplicativos pueden ser importantes. Que un motor de búsqueda tome medio segundo o 2.5 segundos para atender una consulta puede ser la diferencia entre si la gente usa ese motor de búsqueda o va a un competidor.

Por otro lado, al comparar dos algoritmos diferentes, a menudo es el caso que incluso las constantes multiplicativas son irrelevantes. Recuerda que anteriormente examinamos dos algoritmos, enumeración exhaustiva y búsqueda por bisección, para encontrar una aproximación a la raíz cuadrada de un número de punto flotante.

Usando enumeración exhaustiva para aproximar la raíz cuadrada

```
1 def square_root_exhaustive(x, epsilon):
2     """Asume que x y epsilon son floats positivos & epsilon < 1
3     Retorna una y tal que y*y está dentro de epsilon de x"""
4     step = epsilon ** 2
5     ans = 0.0
6     while abs(ans ** 2 - x) >= epsilon and ans * ans <= x:
7         ans += step
8     if ans * ans > x:
9         raise ValueError
10    return ans
```

Usando búsqueda por bisección para aproximar la raíz cuadrada

```
1 def square_root_bi(x, epsilon):
2     """Asume que x y epsilon son floats positivos & epsilon < 1
3     Retorna una y tal que y*y está dentro de epsilon de x"""
4     low = 0.0
5     high = max(1.0, x)
6     ans = (high + low)/2.0
7     while abs(ans ** 2 - x) >= epsilon:
8         if ans**2 < x:
9             low = ans
10        else:
11            high = ans
```

```
12         ans = (high + low) / 2.0
13     return ans
```

Vimos que la enumeración exhaustiva era tan lenta como para ser impráctica para muchas combinaciones de valores para x y ϵ . Por ejemplo, evaluar `square_root_exhaustive(100, 0.0001)` requiere aproximadamente mil millones de iteraciones del bucle `while`. En contraste, evaluar `square_root_bi(100, 0.0001)` toma aproximadamente 20 iteraciones de un bucle `while` ligeramente más complejo. Cuando la diferencia en el número de iteraciones es tan grande, realmente no importa cuántas instrucciones hay en el bucle. Es decir, las constantes multiplicativas son irrelevantes.

Notación Asintótica

Usamos algo llamado notación asintótica para proporcionar una forma formal de hablar sobre la relación entre el tiempo de ejecución de un algoritmo y el tamaño de sus entradas. La motivación subyacente es que casi cualquier algoritmo es suficientemente eficiente cuando se ejecuta con entradas pequeñas. De lo que típicamente necesitamos preocuparnos es de la eficiencia del algoritmo cuando se ejecuta con entradas muy grandes. Como un indicador de "muy grande", la notación asintótica describe la complejidad de un algoritmo cuando el tamaño de sus entradas se aproxima al infinito.

Considera, por ejemplo:

Complejidad asintótica

```
1  def f(x):
2      """Asume que x es un int > 0"""
3      ans = 0
4      #Bucle que toma tiempo constante
5      for i in range(1000):
6          ans += 1
7      print('Number of additions so far', ans)
8      #Bucle que toma tiempo x
9      for i in range(x):
10         ans += 1
11     print('Number of additions so far', ans)
12     #Bucles anidados toman tiempo x**2
13     for i in range(x):
14         for j in range(x):
15             ans += 1
16         ans += 1
17     print('Number of additions so far', ans)
18     return ans
```

Si asumimos que cada línea de código toma una unidad de tiempo para ejecutarse, el tiempo de ejecución de esta función puede ser descrito como $1000 + x + 2x^2$. La constante 1000 corresponde al número de veces que se ejecuta el primer bucle. El término x corresponde al número de veces que se ejecuta el segundo bucle. Finalmente, el término $2x^2$ corresponde al tiempo gastado ejecutando las dos declaraciones en el bucle `for` anidado. En consecuencia, la llamada `f(10)` imprimirá

```
1  Number of additions so far 1000
2  Number of additions so far 1010
3  Number of additions so far 1210
```

y la llamada `f(100)` imprimirá

```
1  Number of additions so far 1000
2  Number of additions so far 2000
3  Number of additions so far 2002000
```

Para valores pequeños de x el término constante domina. Si x es 10, más del 80% de los pasos son explicados por el primer bucle. Por otro lado, si x es 1000, cada uno de los primeros dos bucles cuenta por solo aproximadamente 0.05% de los pasos. Cuando x es 1,000,000, el primer bucle toma aproximadamente 0.0000005% del tiempo total y el segundo bucle aproximadamente 0.0005%. Un total de 2,000,000,000,000 de los 2,000,001,001,000 pasos están en el cuerpo del bucle `for` interno.

Claramente, podemos obtener una noción significativa de cuánto tiempo tomará este código para ejecutarse en entradas muy grandes considerando solo el bucle interno, es decir, el componente cuadrático. ¿Deberíamos preocuparnos por el hecho de que este bucle toma $2x^2$ pasos en lugar de x^2 pasos? Si tu computadora ejecuta aproximadamente 100 millones de pasos por segundo, evaluar `f` tomará aproximadamente 5.5 horas. Si pudiéramos reducir la complejidad a x^2 pasos, tomaría aproximadamente 2.25 horas. En cualquier caso, la moraleja es la misma: probablemente deberíamos buscar un algoritmo más eficiente.

Este tipo de análisis nos lleva a usar las siguientes reglas generales para describir la complejidad asintótica de un algoritmo:

- Si el tiempo de ejecución es la suma de múltiples términos, mantén el que tiene la mayor tasa de crecimiento, y elimina los otros.
- Si el término restante es un producto, elimina cualquier constante.

La notación asintótica más comúnmente usada se llama notación "**Big O**". La notación Big O se usa para dar una cota superior en el crecimiento asintótico (a menudo llamado el orden de crecimiento) de una función. Por ejemplo, la fórmula $f(x) \in O(x^2)$ significa que la función f no crece más rápido que el polinomio cuadrático x^2 , en un sentido asintótico.

Muchos científicos de la computación abusan de la notación Big O haciendo declaraciones como, "la complejidad de $f(x)$ es $O(x^2)$." Con esto quieren decir que en el peor caso f no tomará más de $O(x^2)$ pasos para ejecutarse. La diferencia entre que una función esté "en $O(x^2)$ " y "siendo $O(x^2)$ " es sutil pero importante. Decir que $f(x) \in O(x^2)$ no excluye que el tiempo de ejecución del peor caso de f sea considerablemente menor que $O(x^2)$. Para evitar este tipo de confusión usaremos **Big Theta** (θ) cuando estemos describiendo algo que es tanto una cota superior como una cota inferior en el tiempo de ejecución asintótico del peor caso. Esto se llama una cota ajustada.

Ejercicio manual

¿Cuál es la complejidad asintótica de cada una de las siguientes funciones?

```
1  def g(L, e):
2      """L, una lista de ints, e is un int"""
3      for i in range(100):
4          for el in L:
5              if el == e:
6                  return True
7      return False
8
9  def h(L, e):
10     """L, una lista de ints, e is un int"""
11     for i in range(e):
12         for el in L:
13             if el == e:
14                 return True
15     return False
```

Para `g(L, e)`:

- Peor caso: $O(n)$.
- Mejor caso: $O(1)$.

Para `h(L, e)`:

- Peor caso: $O(e \cdot n)$.

- Mejor caso: $O(1)$.